SELENIUM IDE

---

1. What is Selenium IDE

Selenium IDE is a **record and playback tool** used for:

- Quick automation
- Learning Selenium
- Simple test creation

Key points:

- No programming language required
- Browser extension
- Not suitable for complex frameworks
- Uses **commands instead of code**

---

2. Structure of Selenium IDE Test Step

Each Selenium IDE step has **three parts**:

1. **Command** → What action to perform
2. **Target** → On which element
3. **Value** → Input or expected value

Example:

Command: type
Target: name=q
Value: Selenium

---

3. Types of Selenium IDE Commands

All Selenium IDE commands fall into **three categories**:

1. **Actions**

2. **Accessors**
3. **Assertions**

---

4. ACTION COMMANDS

Actions **interact with the application**.
If an action fails → **test execution stops**.

4.1 open

Opens a URL in browser.

open https://omayo.blogspot.com/

4.2 type

Types text into input field.

type name=q HelloWorld

4.3 send keys

Simulates keyboard keys.

send keys name=q ${KEY_ENTER}

Common key codes:

- KEY_ENTER
- KEY_TAB
- KEY_BACKSPACE
- KEY_DELETE
- KEY_UP / KEY_DOWN

4.4 click

Clicks an element.

click id=checkbox1

4.5 click at

Clicks element at specific coordinates.

click at css=input.btn 10,20

4.6 close

Closes browser window.

close

4.7 submit

Submits a form.

submit css=form[action$='login']

---

5. DROPDOWN COMMANDS

5.1 select

Selects dropdown option.

select id=drop1 label=doc 4

5.2 add selection

Selects option in multi-select dropdown.

add selection id=multiselect1 Volvo

5.3 remove selection

Removes selected option.

remove selection id=multiselect1 Volvo

---

6. CHECKBOX & RADIO COMMANDS

6.1 check

Checks checkbox or radio button.

check id=radio1

6.2 uncheck

Unchecks checkbox.

uncheck id=checkbox1

---

7. ACCESSOR COMMANDS

Accessors **retrieve data** and **store it into variables**.

7.1 store

Stores value into variable.

store title x

7.2 echo

Prints variable value in log.

echo ${x}

7.3 store text

Stores text of element.

store text id=but2 x

7.4 store json

Stores JSON content.

store json { "name":"Ali" } x

---

8. ASSERTION COMMANDS

Assertions **verify application state**.

Forms:

1. **assert** → stops test on failure
2. **verify** → continues test on failure
3. **wait for** → waits for condition

8.1 assert title

Checks page title.

assert title omayo (QAFox.com)

8.2 verify title

Same as assert but continues execution.

verify title omayo (QAFox.com)

8.3 assert text

Checks text of element.

assert text id=but2 Button2

8.4 assert element present

Checks element existence.

assert element present id=but2

8.5 assert checked

Verifies checkbox checked.

assert checked id=checkbox1

8.6 assert editable

Checks field editable state.

assert editable name=q

8.7 assert selected value

Checks selected dropdown value.

assert selected value id=drop1 abc

8.8 assert selected label

Checks selected dropdown visible text.

assert selected label id=drop1 Older Newsletters

8.9 assert alert

Checks alert popup text.

click id=alert1
assert alert Hello

8.10 assert confirm

Checks confirmation dialog.

assert confirm Press OK to confirm

8.11 assert prompt

Checks prompt dialog.

assert prompt What is your name?

---

9. WAIT COMMANDS (IDE)

Used for **dynamic elements**.

9.1 wait for element present

wait for element present id=but2 30000

9.2 wait for element visible

wait for element visible id=delayedText 30000

9.3 wait for element editable

wait for element editable id=dte 30000

---

10. JAVASCRIPT COMMANDS

---

execute script

Executes JavaScript.

execute script alert("Hello World")

---

edit content

Edits contenteditable element.

edit content id=but2 QAFox

Condition:

- Element must have `contenteditable=true`

---

11. CONDITIONAL COMMANDS

11.1 if

if ${x} === "omayo (QAFox.com)"
echo Correct title
end

11.2 if else

```
if ${x} === "abc"
echo Correct
else
echo Wrong
end
```

11.3 else if

```
else if ${x} === "xyz"
```

---

12. LOOP COMMANDS

12.1 while

```
while ${x} < 5
echo ${x}
end
```

Condition checked **before loop**.

12.2 do – repeat if

```
do
echo ${x}
repeat if ${x} < 5
```

Condition checked **after loop**.

12.3 for each

```
for each x itr
echo ${itr}
end
```

Used for arrays.
